

DRIVEAWARE:

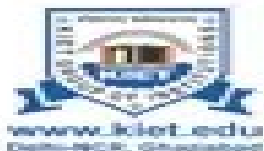
“Because Every Signal Matters”

Project Synopsis
Bachelor of Technology
in
Computer Science and Engineering by

Anshul Saini (Roll No. 2200290100032)
Ayush Chauhan (Roll No. 2200290100049)
Shivam Singh (Roll No. 2200290100152)
Devesh (Roll No. 2200290100059)

Under the Guidance of
Prof. Deepti Singh

Department of Computer Science and Engineering
KIET Group Of Institutions
Dr. APJ Abdul Kalam Technical University, Uttar Pradesh



Abstract

DriveAware play an important major role nowadays in our daily life. Anyway, there are traffic signs on the roads. Despite this, drivers frequently make mistakes. While driving on the road, it is quite difficult to perceive and detect traffic signs. Drivers may misinterpret traffic signs; this leads to Accidents and results in damage to the vehicle.

This project introduces a solution called traffic sign recognition to address a specific issue. The approach involves the utilization of a Convolutional Neural Network (CNN) to extract images and classify various types of traffic signs. These signs encompass a range of categories such as speed limits, no access, traffic lights, turning directions, pedestrian crossings, restrictions for large vehicles, and more. A crucial component of an intelligent transportation system is the implementation of a traffic sign recognition system (TSRS) within the overall framework of an Intelligent Transportation System (ITS). Accurate and efficient recognition of traffic signs can significantly enhance driving safety. Traffic signs possess distinct characteristics, including color and shape, which play a vital role in providing road-related information to drivers. The colors employed in traffic signs across different countries are generally similar, often consisting of primary colors (such as red, blue, yellow) and specific geometric shapes (like circles, triangles, rectangles).

Traffic sign recognition system is implemented using Convolutional Neural Network (CNN). CNN model of this project is for classification. Kaggle German Traffic Sign Recognition Benchmark has been used in the project.

Contents

- Title Page
- Abstract
- Introduction
 - 1.1 Introduction
 - 1.2 Motivation
 - 1.3 Objective and Scope
- Literature Review
- Methodology
- Implementation
- Conclusion
- References

Introduction

DriveAware has indeed emerged as important areas in the field of computer vision, with applications ranging from assisting drivers in understanding and following traffic rules to contributing to the development of autonomous driving systems[1].

One of the challenges in traffic sign detection is the variability in size and shape of the signs. Traffic signs can vary significantly in their dimensions, ranging from small signs to large ones. Additionally, adverse weather conditions like heavy rain or snowfall can obscure the visibility of the signs, making them blurry or partially covered. Overcoming these challenges requires robust detection algorithms that can accurately locate and extract the traffic signs from complex and noisy backgrounds.

Traffic sign categorization is another crucial aspect of traffic sign recognition. Once the traffic signs are detected, the next step is to classify them into specific categories or types, such as speed limits, caution signs, no entry signs, etc.[3] Each category of signs conveys specific information to drivers, and accurate categorization plays crucial role in ensuring the correct interpretation of the signs as well as adherence to traffic rule.

The task of recognising the traffic signs is challenging due to various factors. Firstly, the appearance of traffic signs can vary across different countries and regions, as they may have different shapes, colors, and symbols. This necessitates the development of robust algorithms that can handle the diversity in traffic sign designs. Secondly, the varying lighting conditions and weather conditions can affect the visibility of traffic signs, making their recognition more challenging.

To tackle these challenges, researchers and engineers have developed sophisticated computer vision techniques, including models of deep learning like Artificial Neural Networks (ANNs). CNN has demonstrated excellent performance in image classification tasks, including traffic sign recognition. These models can learn hierarchical features from the traffic sign images and effectively distinguish between different types of signs.

The development and deployment of accurate traffic sign recognition systems have several benefits. For human drivers, such systems can provide real-time information about traffic signs, helping them make informed decisions and improving road safety. In the context of autonomous driving, accurate traffic sign recognition is crucial for ensuring that self-driving vehicles can effectively interpret and respond to traffic rules and regulations.

In conclusion, traffic sign detection and recognition are valuable and challenging research areas within traffic engineering and computer vision. Overcoming the variability in size, shape, and appearance of traffic signs, as well as handling adverse weather conditions, requires robust algorithms and deep learning models[2]. By accurately detecting and categorizing traffic signs, these systems contribute to safer roads along with playing a crucial role in development of intelligent transportation systems and autonomous driving technologies.

1.1.2 Objective and Scope

Traffic Sign Recognition (TSR) systems have gained significant attention in research and development due to their potential to improve driving safety. These systems aim to detect, interpret, and classify traffic signs, providing essential information to drivers and helping them navigate roads more safely. TSR systems are designed to reduce the risk of missing critical traffic signs and overcome human-related issues like fatigue or inattentiveness.

While humans can quickly recognize and interpret traffic signs based on their visual experiences, replicating this process in computers poses challenges. Instead of relying on human experiences, TSR systems leverage the understanding of the characteristics of traffic signs, particularly their color and shape attributes. Although color and shape alone may not be sufficient to distinguish traffic signs from other objects, combining this knowledge with intelligent segmentation techniques can enhance the detection process.

The development of an automatic traffic sign recognition system involves solving several key problems. Firstly, the system needs to detect and locate traffic signs within cluttered scenes, which can include various objects and backgrounds. Detecting traffic signs in real-time and accurately localizing them is crucial for subsequent analysis and interpretation.

Once the traffic signs are detected and located, the system extracts relevant features from the signs. This feature extraction step involves capturing distinct patterns, shapes, and colors that characterize different types of traffic signs. Various computer vision techniques, including edge detection, color histograms, or deep learning-based feature extraction using CNNs, can be employed to extract informative features from the sign images.

After feature extraction, the system performs classification to assign each detected sign to a specific category or type. Classification algorithms, such as support vector machines (SVM), decision trees, or deep neural networks, can be trained on labeled datasets to recognize and classify traffic signs accurately. The availability of large-scale benchmark datasets, like the GTSRB, facilitates training and evaluation of the TSR models.

To enhance real-time performance, TSR systems can leverage advancements in computer vision and machine learning techniques. For instance, integrating TSR with image processing algorithms can enable the detection of traffic signs in video streams or webcam feeds. This enables the system to recognize traffic signals and provide real-time alerts and notifications to drivers.

Implementing TSR systems using webcams or other cameras in vehicles allows for the recognition of traffic signs from the live environment. By continuously monitoring the visual input, the system can detect and interpret traffic signs in real-time, aiding drivers in following traffic rules and regulations[3].

In summary, Traffic Sign Recognition systems that is (TSR) are designed to detect, interpret, and classify traffic signs, enhancing driving safety. These systems rely on the understanding of color and shape attributes of traffic signs, combined with intelligent segmentation and feature extraction techniques. Classification algorithms are employed to assign specific categories to the detected signs. TSR systems can be implemented in real-time scenarios using webcams or video feeds, enabling continuous monitoring and providing timely alerts to drivers.

Literature Review

This should demonstrate a sound understanding of the current state of knowledge in the field, which should critically assess previous and current work in the public domain, drawing out the major facts, views, trends, etc. (not just listing a series of literature/information sources and/or literature/information abstracts).

Traffic sign recognition (TSR) has a rich history and has evolved significantly over the years. Here is an overview of the related work, key milestones, and the evolution of TSR:

1. Early Approaches: In the early days, TSR focused on simple shape-based detection and classification methods. Basic shape recognition algorithms were employed to identify common traffic sign shapes like circles, triangles, and rectangles. These approaches often relied on manual feature engineering and rule-based systems.

2. Template Matching: Template matching techniques were employed, where predefined templates of traffic signs were matched against input images to detect and recognize signs. However,

6

these methods were limited in their robustness to variations in lighting conditions, scales, and rotations.

3. Feature-based Approaches: Feature-based methods emerged as a popular approach in TSR. These methods involved extracting handcrafted features like color, texture, and shape from traffic sign images. Various feature descriptors, such as Histogram of Oriented Gradients (HOGs) were used. Classification algorithms like Support Vector Machines (SVM) or Random Forests were often employed for sign recognition.

4. Machine Learning and Deep Learning: With the rise of machine learning techniques, researchers started exploring the application of these methods in TSR. This led to the adoption of supervised learning algorithms, such as neural networks, for traffic sign classification. Convolutional Neural Networks (CNNs) gained popularity due to their ability to automatically learn hierarchical features from images. Large-scale benchmark datasets like the German Traffic Sign Recognition Benchmark provided a standardized evaluation platform for TSR algorithms.

5. Dataset Creation and Benchmarking: The creation of comprehensive datasets containing a wide variety of traffic sign images played a crucial role in advancing TSR. These datasets, often annotated with sign types and locations, enabled researchers to train and evaluate their models effectively. Benchmarking competitions like the International Joint Conference on Neural Networking (IJCNN) and German Traffic Sign Recognition Competition further spurred progress in the field.

6. Real-time Applications and Integration: TSR systems evolved to operate in real-time scenarios. Integration with onboard cameras, such as webcams or cameras embedded in vehicles, enabled the recognition of the traffic signs directly through live video feeds. The integration allowed for continuous monitoring and immediate feedback to drivers, contributing to enhanced driving safety.

7. Advanced Techniques: As TSR advanced, researchers explored various advanced techniques. This includes domain adaptation to handle variations between different countries' traffic signs, multi-modal fusion using sensor data, and the fusion of TSR with other perception tasks like object detection and pedestrian detection.

8. Autonomous Driving Applications: TSR has found significant applications in autonomous driving systems. It plays a crucial role in enabling self-driving vehicles to perceive and understand traffic signs, aiding in decision-making and ensuring compliance with traffic regulations.

Overall, TSR has evolved from simple shape-based methods to sophisticated deep learning based approaches. The development of benchmark datasets, advancements in ML algorithms, and integration with real-time system have significantly contributed to the progress in TSR. Continued research and development in this field are essential for further improving driving safety and enabling autonomous vehicles to navigate roads effectively.

There have been numerous notable works in the field of traffic sign recognition (TSR). While it is challenging to pinpoint a single best work, we can mention some influential and highly regarded research papers and projects in TSR:

1. "TSR with Multi-Scale Convolutional Networks" by Sermanet et al. (2011): This pioneering work applied Convolutional Neural Networks (CNNs) to TSR and introduced the concept of multi-scale CNNs for robust detection along with the classification of traffic sign. It demonstrated impressive performance on the German Traffic Sign Recognition Benchmark dataset.
2. "Deep Neural Networks for Traffic Sign Recognition" by Cireşan et al. (2012): The following paper proposed a deep learning architecture that is based on CNNs for traffic sign recognition. It has got state-of-the-art results on various traffic signs datasets, including the GTSRB.
3. "Traffic Sign Recognition using Deep Hierarchical Network" by Zhu et al. (2016): This work introduced a deep hierarchical network for TSR that leveraged both global and local information of traffic signs. The model achieved excellent performance on various benchmark datasets and demonstrated robustness to occlusions and variations in lighting conditions.
4. "End-to-End Multi-Task Learning with Attention for Traffic Sign Recognition" by Zhang et al. (2019): This research paper presented an end-to-end multi-task learning framework with attention mechanisms for TSR. It effectively combined detection and classification tasks and introduced a kind of attention mechanism to focus upon important regions of traffic signs. The proposed model achieved competitive performance on the German Traffic Sign Recognition Benchmark.

5. "Traffic Sign Detection and Classification in the Wild" by Houben et al. (2020): This work focused on traffic sign detection and classification in real-world scenarios, including challenging conditions like adverse weather, occlusions, and low lighting. The authors proposed an integrated approach combining deep learning models, data augmentation, and ensemble learning to achieve robust performance on a large-scale real-world dataset.

It is worth noting that TSR is a rapidly evolving field, and new research and advancements continue to emerge. The best work may vary depending on the specific evaluation criteria, dataset, and application context. Nonetheless, the aforementioned studies have made significant contributions to the development of TSR and have greatly influenced the research community.



Figure 2.1: Traffic Sign of GTSRB

In Figure 2.1: Traffic Sign of GTSRB having different traffic signs has been represented.

Methodology

3.1 Methodology

This chapter presents the methodology used in conjunction with various modeling techniques, including non-parametric, semi-parametric, and parametric models.

3.1.1 CNN

A typical architecture of a CNN contains an input layer, some convolutional layers, some fully connected layers, and an output layer. CNN is a multi-layered network that functions similarly

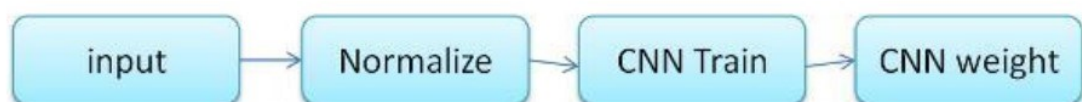


Figure 3.1: Training Phase

In Figure 3.1: A typical CNN architecture's training phase is shown.

12

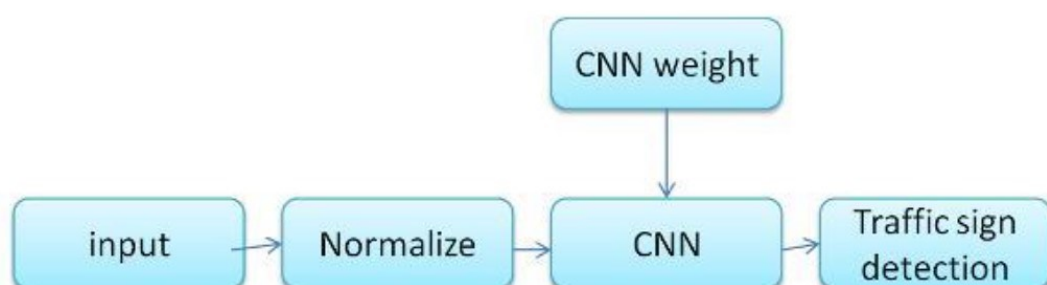


Figure 3.2: Testing Phase

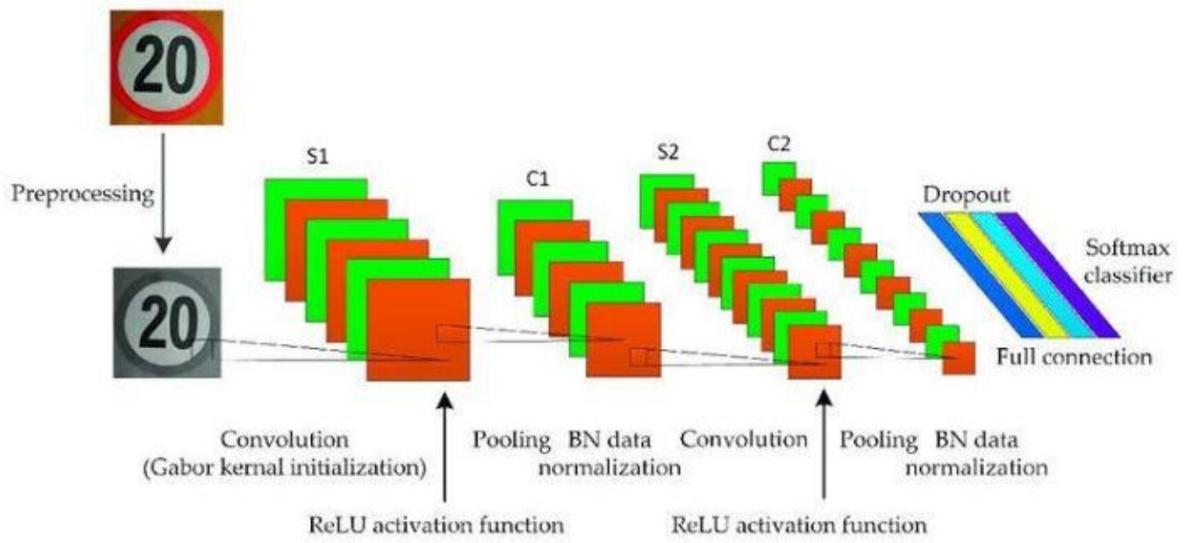


Figure 3.4: Using dropout with CNN

In Figure 3.5: Generating new transformed versions of images from the given image dataset to increase its diversity.

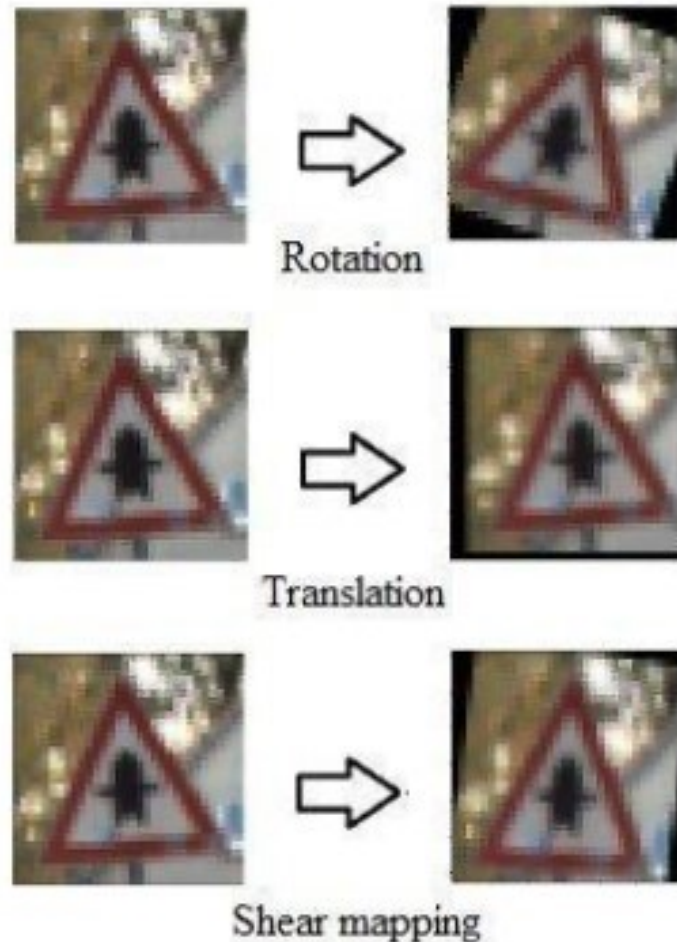


Figure 3.6: Result After Applying Data Augmentation
In Fig 3.6: Results obtained after application of data augmentation

3.4 Steps for Traffic Sign Recognition

In the proposed system, Traffic sign detection and recognition has been achieved by CNN algorithm. Prior to classification, the input undergoes pre-processing to eliminate noise and enhance the quality of the data, reduce complexity, and increase the precision of the employed method. In any TSR application, the detection of traffic sign is critical. In fact, an incorrectly detected



Figure 3.7: GTSRB Dataset

In Figure 3.7: The world renowned German Traffic Sign Recognition Benchmark

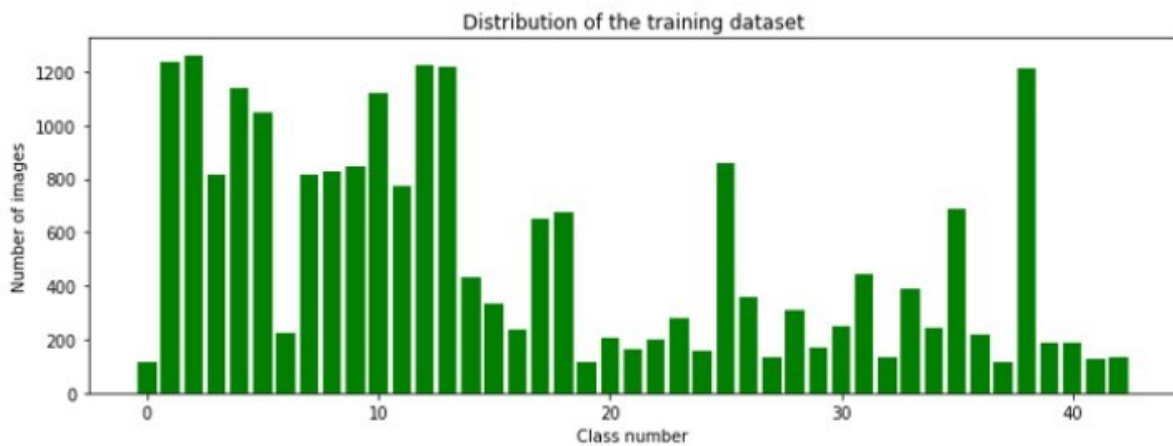


Figure 3.8: Distribution of the training dataset

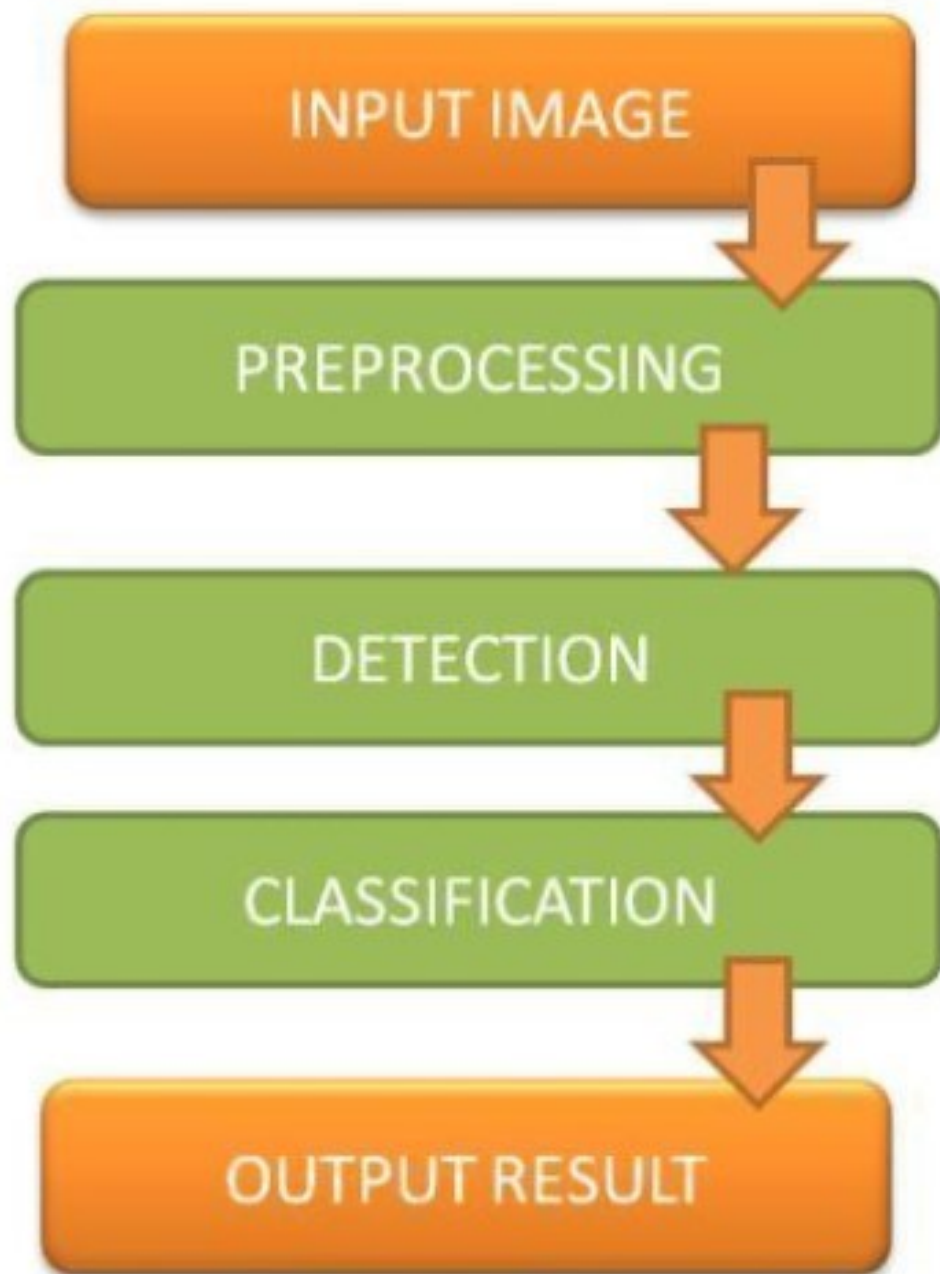
In Figure 3.8: Distribution of the training dataset and segregation into 43 classes

sign cannot be categorized and recognized in order to alert the motorist. The distinctive properties of traffic signs must be examined throughout the detection process. The classification module receives the detected regions of interest (ROIs) and assigns them to one of the predefined classes, which include danger, information, obligation or prohibition, or identifies them as non-signs.

Implementation

- The convolutional neural network (CNN) is a particular class of deep learning neural networks.
- CNNs represent a big breakthrough in image recognition.
- They are normally used to analyse visual imagery and frequently works behind the scenes in image classification.
- CNNs in reality use very less pre-processing, this means that they can also learn the filters that have to be manually-made in other algos.
- A CNN working principle is based on extraction of features from images. This eliminates the need of manual feature extraction.
- The features are not trained. They're learned while the network trains on the set of images.

CNN architecture for Traffic Sign recognition as mentioned above was implemented in Python. Along with Python programming language, NumPy, Keras, tensorflow and OPENCV libraries



```

#Making model
model = keras.models.Sequential([
    keras.layers.Conv2D(filters=16, kernel_size=(3,3), activation='relu', input_shape=(IMG_HEIGHT,IMG_WIDTH,channels)),
    keras.layers.Conv2D(filters=32, kernel_size=(3,3), activation='relu'),
    keras.layers.MaxPool2D(pool_size=(2, 2)),
    keras.layers.BatchNormalization(axis=-1),

    keras.layers.Conv2D(filters=64, kernel_size=(3,3), activation='relu'),
    keras.layers.Conv2D(filters=128, kernel_size=(3,3), activation='relu'),
    keras.layers.MaxPool2D(pool_size=(2, 2)),
    keras.layers.BatchNormalization(axis=-1),

    keras.layers.Flatten(),
    keras.layers.Dense(512, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(rate=0.5),

    keras.layers.Dense(43, activation='softmax')
])

```

Python

Figure 3.11: Building of Model
In Figure 3.11: Codebase for model building is shown.

```

Epoch 1/25
858/858 [=====] - 108s 123ms/step - loss: 1.0998 - accuracy: 0.7016 - val_loss: 0.1106 - val_accuracy: 0.9663
Epoch 2/25
858/858 [=====] - 116s 135ms/step - loss: 0.2145 - accuracy: 0.9337 - val_loss: 0.0317 - val_accuracy: 0.9917
Epoch 3/25
858/858 [=====] - 96s 112ms/step - loss: 0.1217 - accuracy: 0.9617 - val_loss: 0.0215 - val_accuracy: 0.9931
Epoch 4/25
858/858 [=====] - 91s 106ms/step - loss: 0.1112 - accuracy: 0.9658 - val_loss: 0.0262 - val_accuracy: 0.9932
Epoch 5/25
858/858 [=====] - 88s 103ms/step - loss: 0.0848 - accuracy: 0.9730 - val_loss: 0.0315 - val_accuracy: 0.9904
Epoch 6/25
858/858 [=====] - 87s 101ms/step - loss: 0.0799 - accuracy: 0.9764 - val_loss: 0.0216 - val_accuracy: 0.9940
Epoch 7/25
858/858 [=====] - 86s 101ms/step - loss: 0.0654 - accuracy: 0.9806 - val_loss: 0.0125 - val_accuracy: 0.9963
Epoch 8/25
858/858 [=====] - 87s 101ms/step - loss: 0.0602 - accuracy: 0.9820 - val_loss: 0.0181 - val_accuracy: 0.9957
Epoch 9/25
858/858 [=====] - 87s 101ms/step - loss: 0.0526 - accuracy: 0.9845 - val_loss: 0.0249 - val_accuracy: 0.9927
Epoch 10/25
858/858 [=====] - 89s 104ms/step - loss: 0.0492 - accuracy: 0.9842 - val_loss: 0.0186 - val_accuracy: 0.9948
Epoch 11/25
858/858 [=====] - 89s 103ms/step - loss: 0.0500 - accuracy: 0.9848 - val_loss: 0.0297 - val_accuracy: 0.9922
Epoch 12/25
858/858 [=====] - 90s 104ms/step - loss: 0.0435 - accuracy: 0.9862 - val_loss: 0.0129 - val_accuracy: 0.9963
Epoch 13/25
...
Epoch 24/25
858/858 [=====] - 106s 123ms/step - loss: 0.0252 - accuracy: 0.9920 - val_loss: 0.0064 - val_accuracy: 0.9980
Epoch 25/25
858/858 [=====] - 102s 119ms/step - loss: 0.0256 - accuracy: 0.9922 - val_loss: 0.0246 - val_accuracy: 0.9922

```

Figure 3.12: Training of Model In Figure 3.12:
Codebase for model training is shown.

were used. Train and validate the model after building the model architecture, we then train the model using model. Fit (). I tried with batch size 32 . And after 10 epochs the accuracy was almost stable. Our model got a 98.8% accuracy on the training dataset. With matplotlib, we plot the graph for accuracy and the loss.

The CNN classifier is employed to classify images captured from the laptop's webcam. Initially, openCV is utilized to detect traffic signs within each frame of the webcam feed. The region of the image containing the sign is then resized to a size of 32x32 and served as input to the CNN. The network generates a list of softmax scores corresponding to the 43 classes of traffic signs. The sign with the highest probability is displayed on the screen. Below are a few examples showcasing the classification results obtained using the webcam.



Figure 3.13: Output Detection

In Figure 3.13: Output is detected with Class 0 which is Speed Limit - 20km/hr

Conclusion

5.1 Conclusion

The proposed system for traffic sign classification and detection appears to be effective in handling various challenges such as blurry signs, small sizes, and improper display conditions. The system follows a multi-step process to achieve accurate recognition and classification.

The first step involves pre-processing the input image. Pre-processing techniques can include image enhancement, noise reduction, and normalization to improve the quality and standardize the input images for further analysis.

Next, the system focuses on traffic sign detection. Shape features are utilized to identify potential traffic signs in the image. Shape-based detection techniques can involve edge detection, contour analysis, or template matching to locate regions that resemble common traffic sign shapes like circles, triangles, or rectangles. By leveraging shape information, the system aims to identify the presence and location of the traffic signs in an image.

References

- [1] Njyou Youssouf, 2022. "Traffic sign classification using CNN and detection using fasterRCNN and YOLOV4". Department of Computer Science and Engineering, Islamic University of Technology.
- [2] Jiawei Xing¹, Minh Nguyen¹, Wei Qi Yan, 2022. "The Improved Framework for Traffic Sign Recognition Using Guided Image Filtering ". Department of Computer Science and Engineering, Islamic University of Technology. SN Computer Science (2022) 3:461
- [3] Rajesh Kannan Megalingam, Kondareddy Thanigundala, Sreevatsava Reddy Musani, Hemanth Nidamanuru, Lokesh Gadde, 2022. "Indian traffic sign detection and recognition using deep learning". Department of Electronics and Communication Engineering, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Amritapuri, Kerala, India
- [4] Omar Sayed, Vishal kumar pal, and khisty jassal, *Recognition of Traffic Sign*, 2019
- [5] Md Tarequl Islam, "Traffic sign detection and recognition based on convolutional neural networks", International Conference on Advances in Computing, Communication and Control (ICAC3), DOI: 10.1109/ICAC347590.2019.9036784 2019 .
- [6] Prashengit Dhar , Md. Zainal Abedin , Tonoy Biswas and Anish Datta, *Traffic Sign Detection- A New Approach and Recognition Using Convolutional Neural Network* 2017
- [7] Sanam Narejo, Shahnawaz Talpur, Madeha Memon, and Amna Rahoo *TAn Automated System for Traffic sign Recognition using Convolutional Neural Network*, 2020
